

JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 6 OF 18

Arrays and Array Problem-Solving Patterns

Traversal, In-Place Work, Pointers, Windows, Prefix Sums, and Sorting

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Build a reusable array pattern map from storage semantics through two pointers, sliding windows, prefix sums, sorting, and list trade-offs.

ARRAYS | TWO POINTERS | WINDOWS | PREFIX SUMS | SORTING | LISTS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to **05 – Loops and Index Calculations**. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Arrays are the default interview state space. This volume joins Java representation rules with the pattern-selection reasoning needed for efficient solutions.

Readiness map

Decision	Evidence
START HERE	Contiguous ranges or ordered positions matter; In-place mutation can save auxiliary space; A window changes incrementally; Many range queries suggest prefix state.
PREPARE FIRST	Volumes 1-5; Comfort with loops and complexity.
MOVE ON WHEN	Use arrays without aliasing or copy mistakes; Select two-pointer, window, prefix, difference, or sorting patterns; Explain ArrayList trade-offs; Test duplicates, empty ranges, and boundary indexes.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

05 - Loops and Index Calculations	YOU ARE HERE 06 - Arrays and Array Patterns	07 - Strings and String Patterns
-----------------------------------	---	----------------------------------

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Java Array Semantics	4
Chapter 2 - ArrayList and List Trade-offs	8
Chapter 3 - Sorting Arrays Safely	14
Chapter 4 - SDE-2 Array Problem-Solving Patterns	17
Part II - Practice and Handoff	37
Practice Ladder and Next Steps	37

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Java Array Semantics

WHY IT WORKS | First-principles model

An array is an object with a runtime component type and immutable length. Its elements are mutable unless prevented by external design. A variable such as `int[] values` contains a reference, not the elements themselves. Multidimensional array syntax represents arrays whose elements are array references; rows may have different lengths or be null.

A `String` is an immutable sequence of UTF-16 code units. A `char` is one 16-bit code unit. Some Unicode code points use one code unit, others use a surrogate pair, and a visible grapheme can combine several code points. Therefore `length()`, code-point count, and what a user sees are different concepts.

A text block is source syntax for a `String`. It improves multiline readability but does not introduce a different runtime type.

Specification boundary: Java specifies array bounds checks, runtime store checks, `String` behavior, and UTF-16-based APIs. It does not guarantee a particular internal `String` field layout or byte encoding used inside a JVM implementation.

Core terminology

- **Component type:** type of an array's elements.
- **Reified:** runtime retains enough component-type information to check array stores.
- **Covariance:** `Sub[]` is a subtype of `Super[]` when `Sub` is a subtype of `Super`.
- **Aliasing:** multiple references reach the same mutable object.
- **Shallow copy:** copies top-level values or references, not referenced nested objects.
- **Interning:** canonicalizing equal strings in a JVM-managed pool.
- **Code unit:** unit used by an encoding; Java `char` is a UTF-16 code unit.
- **Code point:** Unicode numeric value such as U+1F680.
- **Grapheme cluster:** sequence perceived as one user-visible character.
- **Incidental indentation:** whitespace text blocks remove based on delimiter and content placement.

Detailed mechanics

Array creation and initialization

JAVA

```
int[] counts = new int[4];           // [0, 0, 0, 0]
String[] names = {"Ada", "Linus"};
int[][] matrix = new int[3][];
matrix[0] = new int[] {1, 2};
matrix[1] = new int[] {3};
// matrix[2] remains null
```

Every array element is default-initialized. Length is available through the final-like `length` field and cannot change. Indexes range from zero through `length - 1`; any other index throws `ArrayIndexOutOfBoundsException`.

Array declarations allow brackets after the type or name, but placing them with the type avoids mixed declarations. Generic array creation such as `new List<String>[10]` is illegal because erased generic element types cannot support the array's runtime store check.

Covariance is checked dynamically:

JAVA

```
Number[] numbers = new Integer[2];
numbers[0] = 10;
// numbers[1] = 2.5; // ArrayStoreException
```

The variable permits a `Double` by its static component type, but the actual object is `Integer[]`, so the store fails. Generic collections use invariant static checking and usually make this class of error impossible.

Copying and comparison

Assignment copies an array reference. `clone`, `Arrays.copyOf`, and `System.arraycopy` create or populate top-level arrays, but nested references remain shared.

JAVA

```
int[][] original = {{1, 2}, {3, 4}};
int[][] shallow = original.clone();
shallow[0][0] = 99;
System.out.println(original[0][0]); // 99
```

Arrays inherit identity-based `equals` and `hashCode` from `Object`. Use `Arrays.equals` for one-dimensional content, `Arrays.deepEquals` for nested arrays, and matching hash helpers. Printing an array directly does not format its elements; use `Arrays.toString` or `deepToString`.

TRACE IT | Worked Java example

This function reverses code points rather than UTF-16 code units and preserves supplementary characters.

JAVA

```
public class UnicodeReverse {
    static String reverseCodePoints(String input) {
        int[] points = input.codePoints().toArray();
        StringBuilder result = new StringBuilder(input.length());
        for (int i = points.length - 1; i >= 0; i--) {
            result.appendCodePoint(points[i]);
        }
        return result.toString();
    }

    public static void main(String[] args) {
        String input = "A\uD83D\uDE80B";
        System.out.println(reverseCodePoints(input)); // B, rocket, A
    }
}
```

This is code-point correct, not grapheme-cluster correct. Reversing combining marks separately can still produce surprising visible output. User-perceived text segmentation requires a boundary algorithm suitable for the Unicode version and locale.

TRACE IT | Execution or memory walkthrough

The input contains four UTF-16 code units: **A**, a high surrogate, a low surrogate, and **B**. `codePoints()` combines the surrogate pair and emits three `int` values. `toArray` allocates an `int[3]`.

The loop begins at index two and appends **B**. It then calls `appendCodePoint` for the rocket value, which emits two UTF-16 code units, followed by **A**. `toString` creates the immutable result. The original string is unchanged.

No bounds error occurs because the loop starts at `points.length - 1`, continues while nonnegative, and decrements. For an empty input it starts at -1 and performs zero iterations.

Complexity and performance

Array indexing is $O(1)$; traversal and copying are $O(n)$. `System.arraycopy` is still $O(n)$, though JVMs optimize its constant factors. A rectangular r by c traversal is $O(rc)$; a jagged traversal is proportional to actual elements.

Most string searches and transformations are $O(n)$ in code units, with algorithm-specific exceptions. Concatenating into an immutable accumulator inside a loop can be $O(n^2)$ in total copied content. A pre-sized `StringBuilder` usually makes assembly $O(n)$.

The reverse example is $O(n)$ time and $O(n)$ additional space. A more intricate backward code-point traversal can avoid the `int[]`, but clarity is often worth the allocation unless profiling identifies the path.

HotSpot note: Current HotSpot implementations may store strings compactly using a byte array plus an encoding marker when contents permit. This is not a public `String` contract and should not influence correctness assumptions.

WATCH OUT | Edge cases and common mistakes

- Array assignment aliases; it does not copy elements.
- `clone` on a nested or object array is shallow.
- Covariant array stores can fail at runtime.
- `Arrays.asList(primitiveArray)` creates a one-element list containing that array, not a list of boxed primitives.
- `array.length`, `string.length()`, and `collection.size()` use different syntax.
- `==` compares string references; `equals` compares content.
- Calling `toString` on a null reference fails; `String.valueOf` renders it as `"null"`, which may hide missing data.
- Splitting on a regex metacharacter requires regex escaping, and `split` discards trailing empty strings by default unless a negative limit is used.
- A supplementary code point occupies two `char` values.
- Code-point correctness does not guarantee grapheme or locale correctness.
- Default-charset conversions vary by environment; specify `StandardCharsets.UTF_8` at byte boundaries.
- Text blocks still process escapes and normally include a trailing newline based on delimiter placement.

TRY IT | Exercises

- 1 Deep-copy a jagged `int[][]` while preserving null rows.
- 2 Demonstrate an `ArrayStoreException`, then replace the API with a generic collection.
- 3 Write tests showing trailing-newline and indentation behavior for three text blocks.
- 4 Count code units, code points, and expected grapheme clusters in strings containing an emoji and a combining mark.
- 5 Implement UTF-8 encode/decode with a `CharsetEncoder` configured to report malformed input.
- 6 Replace quadratic loop concatenation with a pre-sized `StringBuilder` and justify the capacity estimate.

SDE-2 CORE CHAPTER

Chapter 2 - ArrayList and List Trade-offs

WHY IT WORKS | First-principles model

A `List<E>` represents an ordered sequence indexed from `0` through `size - 1`. It permits duplicates and, depending on implementation, may permit null. Equality compares corresponding elements in sequence.

An array-backed list maintains conceptually:

TEXT

```
elements: [e0, e1, e2, e3, null, null, null, null]
size:      4
capacity:  8
```

```
Invariant: 0 <= size <= capacity
Valid logical elements occupy indexes [0, size).
```

A doubly linked list maintains nodes:

TEXT

```
null <- [prev|A|next] <-> [prev|B|next] <-> [prev|C|next] -> null
      first                                     last
```

```
Invariant: first.prev == null, last.next == null,
and adjacent next/prev links agree.
```

The dynamic array makes location computation cheap: address is derived from base plus index. The linked representation must follow references to find an index. Once a linked node is already known, relinking neighbors is constant work, but the public `List` API usually supplies an index or search value, not a node handle.

Specification boundary: `List` promises positional behavior, not storage representation or complexity. `ArrayList` documents constant-time positional access and amortized constant-time append. Its exact capacity-growth policy is not a public contract.

Core terminology

- **Logical size:** Number of elements visible through the list.
- **Capacity:** Number of slots currently available in an array-backed representation.
- **Amortized cost:** Average cost per operation over a sequence, including occasional expensive operations.
- **Random access:** Efficient access by arbitrary index. `RandomAccess` is a marker used by some algorithms to adapt behavior.
- **Locality:** Tendency for nearby data to be located near each other in memory and used together.
- **Node:** A linked-list record containing an element and neighbor references.
- **Cursor:** A position between elements, represented by `ListIterator`.
- **Range view:** A live portion of a list, typically from `subList`.
- **Fixed-size list:** Element replacement is allowed, but size-changing operations are rejected.

Detailed mechanics

ArrayList operations

`get(i)` and `set(i, value)` validate the index and access one slot. Appending writes into the next slot if capacity remains. When full, the implementation allocates a larger array and copies references. Inserting at index `i` shifts the suffix `[i, size)` one slot right. Removing at `i` shifts the later suffix left and clears the obsolete final slot so the removed object is not retained through that array reference.

Suppose size equals capacity and append triggers a copy of `n` references. One append is $O(n)$, but if capacity grows geometrically, many preceding appends were cheap. Across a long append sequence, total copied references form a geometric series bounded by a constant multiple of `n`, yielding amortized $O(1)$ append.

`ensureCapacity` can reduce repeated resizing when a reasonable size estimate exists. `trimToSize` can reduce spare storage but may cost $O(n)$ and cause future regrowth. Neither should be called reflexively.

HotSpot note: OpenJDK has historically grown `ArrayList` capacity by roughly one-half of the old capacity in common cases, with boundary handling. This is version-sensitive implementation detail. Never write logic that depends on a particular capacity sequence.

Iteration and ListIterator

For both structures, one full iterator traversal is $O(n)$. Indexed traversal can be disastrous for a linked list:

JAVA

```
for (int i = 0; i < list.size(); i++) {
    consume(list.get(i));
}
```

If `list` is a `LinkedList`, repeated `get` makes the loop $O(n^2)$. An enhanced `for` loop or iterator is $O(n)$.

`ListIterator` supports forward and backward traversal, reports neighboring indexes, and can add, remove, or replace at its cursor when supported. Its state rules matter: `remove` or `set` must follow a successful `next` or `previous`, and cannot immediately follow `add` or another `remove`.

Typical general-purpose list iterators are fail-fast when they detect structural mutation outside the iterator. Detection is best-effort and must not be used for synchronization or program control. Use the iterator's mutation methods or establish exclusive ownership.

List construction variants

- `new ArrayList<>()` is mutable and resizable.
- `new ArrayList<>(source)` is a shallow mutable copy.
- `Arrays.asList(array)` is a fixed-size view backed by the array. `set` changes the array.
- `List.of(elements...)` is unmodifiable and rejects null.
- `List.copyOf(source)` is unmodifiable and rejects null; it may reuse an already suitable immutable instance.
- `Collections.unmodifiableList(source)` is an unmodifiable view backed by `source`.
- `stream.toList()` returns an unmodifiable list, but code should not assume a concrete implementation.

Sublists

`list.subList(from, to)` uses a half-open range: `from` is included and `to` is excluded. It is normally backed by the parent. Clearing a sublist can efficiently remove a range from the parent. Structural modification of the backing list outside the view invalidates assumptions; subsequent behavior is generally not useful to depend upon and commonly produces `ConcurrentModificationException`.

If a stable independent range is needed, use `new ArrayList<>(list.subList(from, to))`.

Conversion to arrays

`toArray()` returns `Object[]`. `toArray(new String[0])` and `toArray(String[]::new)` produce a typed array. Modern JVMs optimize common zero-length-array idioms, so prefer clarity and measure if this conversion is actually hot. Conversion copies references and costs $O(n)$ space and time.

TRACE IT | Worked Java example

This example keeps a sorted timeline in an `ArrayList` and inserts a new event using binary search:

JAVA (continued 1/2)

```
import java.time.Instant;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

record Event(Instant at, String id) {}

final class Timeline {
    private static final Comparator<Event> ORDER =
        Comparator.comparing(Event::at).thenComparing(Event::id);

    private final ArrayList<Event> events = new ArrayList<>();

    void add(Event event) {
        int result = java.util.Collections.binarySearch(events, event, ORDER);
        int insertionPoint = result >= 0 ? result : -result - 1;
        events.add(insertionPoint, event);
    }
}
```

JAVA (continued 2/2)

```
List<Event> between(Instant startInclusive, Instant endExclusive) {
    ArrayList<Event> result = new ArrayList<>();
    for (Event event : events) {
        if (event.at().isBefore(startInclusive)) {
            continue;
        }
        if (!event.at().isBefore(endExclusive)) {
            break;
        }
        result.add(event);
    }
    return List.copyOf(result);
}

List<Event> snapshot() {
    return List.copyOf(events);
}
}
```

The list invariant is that `events` is sorted by `ORDER`. The class, not callers, owns mutation. A real system with frequent arbitrary insertion may choose a tree index or append plus periodic sort; the example illustrates how search cost and movement cost differ.

Complexity and performance

Operation	<code>ArrayList</code>	<code>LinkedList</code>
get or set by index	$O(1)$	$O(n)$
append	amortized $O(1)$	$O(1)$
add or remove at front	$O(n)$	$O(1)$
insert/remove at arbitrary index	$O(n)$ shift	$O(n)$ search plus $O(1)$ relink
insert/remove through positioned iterator	$O(n)$ shift	$O(1)$ relink
search by value	$O(n)$	$O(n)$
full iteration	$O(n)$	$O(n)$
auxiliary structure per element	array slot	node plus links

The table hides hardware effects. `ArrayList` traversal is usually faster because reference slots are contiguous, copying can use optimized bulk operations, and there are fewer allocations and indirections. `LinkedList` nodes increase garbage collection work and can miss CPU caches.

For small lists, constants dominate. For very large lists, the `int` index and array-size limits matter. Neither implementation should be treated as a disk-scale container. Benchmark representative data with the intended JDK and hardware.

WATCH OUT | Edge cases and common mistakes

- Using `LinkedList` for frequent indexed reads, creating accidental quadratic behavior.
- Claiming linked insertion is $O(1)$ while ignoring the $O(n)$ search for a position.
- Using `remove(1)` on `List<Integer>` when intending to remove the value `1`; it removes index `1`. Use `remove(Integer.valueOf(1))` for value removal.
- Assuming `Arrays.asList` is a normal resizable `ArrayList`.
- Exposing `subList` as a durable independent result.
- Forgetting half-open range boundaries and accepting `from > to` or invalid indexes.
- Retaining a tiny sublist backed by a very large custom or older representation; make an independent copy when retention is uncertain.
- Sorting a list and later mutating fields used by ordering without restoring the invariant.
- Treating `List.of(array)` as a list of array elements when a primitive array is passed; a primitive array is one reference element.
- Forgetting null rejection in immutable factories.
- Relying on exact `ArrayList` capacity growth.
- Using copy-on-write lists for write-heavy workloads; that concurrent implementation copies on mutation.

TRY IT | Exercises

- 1 Dry-run removal at index `2` from an array list of six elements. Count moved references and identify the slot that must be cleared.
- 2 Prove that indexed traversal of a linked list is $O(n^2)$ by summing traversal distances.
- 3 Implement a sorted list insertion method that places a duplicate after all equal values. State why binary search alone does not promise which equal item it finds.
- 4 Demonstrate aliasing between an array and `Arrays.asList(array)`. Contrast it with `new ArrayList<>(...)`.
- 5 Design a benchmark hypothesis comparing `ArrayList`, `LinkedList`, and `ArrayDeque` for removing from the front. Include allocation and warmup considerations.
- 6 Refactor a class that returns its mutable internal list into an ownership-safe API while preserving deterministic order.

SDE-2 CORE CHAPTER

Chapter 3 - Sorting Arrays Safely

Detailed mechanics

Stability and tie-breaking

Stable sort preserves input order when comparator returns zero. This permits multi-pass sorting: stable-sort by a secondary key, then by primary key. A single comparator chain is usually clearer and performs fewer sorts.

Stability is not a substitute for a deterministic total presentation order. If input arrives from a hash map or concurrent source with unspecified encounter order, stable sorting equal ranks preserves an unspecified order. Add a unique immutable ID as a tie-breaker for reproducible pagination and tests.

Library sorting APIs

`List.sort(comparator)` mutates the list and is specified as stable. `Collections.sort` delegates to the list's sorting facility in modern APIs. The list must support the required replacement operation; an unmodifiable list rejects sorting.

`Arrays.sort(Object[])` is stable. Primitive overloads are not required to be stable because primitive values have no separately observable identity when equal. Primitive sorts avoid boxing and are normally much more memory-efficient.

`Arrays.parallelSort` may use the common fork-join pool and temporary storage. Parallel overhead can outweigh gains for small arrays or contended services. Measure on the deployment shape, and remember that comparator work must be thread-safe and side-effect-free.

HotSpot note: Current OpenJDK releases commonly use TimSort-derived logic for object arrays and lists, dual-pivot quicksort and specialized paths for several primitive types, and parallel merge/sort strategies above thresholds. Algorithms and thresholds are version-sensitive.

TimSort intuition and comparator failures

TimSort is adaptive: it discovers ascending or descending runs, extends short runs, and merges runs while maintaining stack invariants. Nearly sorted input can therefore be efficient. It needs temporary storage for merging.

Inconsistent comparators can trigger exceptions such as "comparison method violates its general contract" in some sort paths, but detection is not guaranteed. A sort that appears to complete does not prove comparator correctness.

Selection instead of full sorting

If only the smallest or largest k items are needed:

- sort all: $O(n \log n)$ time, straightforward, produces complete order;
- maintain a heap of size k : $O(n \log k)$ time and $O(k)$ extra space;
- quickselect: expected $O(n)$ to place the k th rank, then optionally sort the selected k ; worst case $O(n^2)$ without robust pivot strategy;
- counting/bucket techniques: potentially $O(n + \text{range})$ when integer key range is small and bounded.

Quickselect partitions around a pivot. After partition, if the pivot lands at rank k , selection is complete; otherwise recurse or iterate only on the side containing rank k . It mutates the array unless implemented on a copy.

Complexity and performance

For **topK**, each of n inputs performs constant comparison plus at most one heap replacement costing $O(\log k)$. Time is $O(n \log k + k \log k)$ and auxiliary reference storage is $O(k)$. When $k \geq n$, the behavior approaches $O(n \log n)$ and sorting all may be simpler and faster.

Common sorting bounds:

Technique	Time	Extra space	Stable	Best use
comparison sort	$O(n \log n)$ typical/guaranteed by chosen algorithm	varies	API-dependent	full order
insertion sort	$O(n^2)$, near $O(n)$ when nearly sorted	$O(1)$	yes when implemented conventionally	tiny/nearly sorted ranges
merge sort	$O(n \log n)$	$O(n)$	yes	stable predictable sorting
heap sort	$O(n \log n)$	$O(1)$ array model	no	worst-case bound, low extra space
quicksort	average $O(n \log n)$, worst $O(n^2)$	recursion/stack varies	usually no	fast primitive/in-place-style sorting
heap top-k	$O(n \log k)$	$O(k)$	only with explicit tie order	small top subset
quickselect	expected $O(n)$, worst $O(n^2)$	often $O(1)$ iterative	no	one rank or unsorted partition

Comparator cost can dominate $n \log n$. Avoid network calls, database access, mutable clocks, locale creation, or repeated expensive parsing inside comparison. Precompute keys when profiling justifies it.

WATCH OUT | Edge cases and common mistakes

- Implementing compare with subtraction and overflowing.
- Omitting a stable tie-breaker for pagination or distributed merge results.
- Assuming stable sort makes unspecified input order deterministic.
- Using a comparator that changes based on mutable state, current time, or side effects.
- Mixing natural and external ordering between sort and binary search.
- Expecting binary search to return the first duplicate.
- Sorting an unmodifiable or fixed-capability list without understanding supported operations.
- Mutating comparator fields while objects reside in a sorted collection.
- Assuming primitive and object sort stability are identical.
- Parallelizing a small sort or using a non-thread-safe comparator with parallel sort.
- Fully sorting millions of items to return a tiny top-k result.
- Forgetting that `reversed()` placement can reverse an entire comparator chain.
- Treating current OpenJDK algorithm names or thresholds as portable guarantees.

TRY IT | Exercises

- 1 Write a comparator for nullable invoices ordered by status, descending amount, due date, and unique ID. State null policy.
- 2 Produce a three-value counterexample showing how a bad cyclic comparator violates transitivity.
- 3 Implement lower-bound binary search that returns the first index whose element is not less than a target.
- 4 Compare full sort, heap top-k, and quickselect for $n = 1,000,000$ and $k = 10$, then for $k = 900,000$.
- 5 Repair a comparator written as `(a, b) -> a.timestamp() > b.timestamp() ? 1 : -1`.
- 6 Explain why stable sorting entries from a `HashMap` by value alone can still produce changing output.

SDE-2 CORE CHAPTER

Chapter 4 - SDE-2 Array Problem-Solving Patterns

Why arrays are the interview default

An array is a fixed-size, indexed region of homogeneous references or primitive values. That simple model supports a large fraction of interview problems because position, order, and contiguous ranges are explicit. The hard part is rarely the loop itself. The hard part is recognizing which information can be maintained incrementally, whether mutation is allowed, and which invariant makes the result trustworthy.

At SDE-2 level, a candidate should move beyond "use two pointers" and explain why that pattern applies, which region of the array has a settled meaning, how overflow and aliasing are handled, and what would change in a production API.

Learning objectives

After completing this chapter, you should be able to:

- select an array pattern from ordering, contiguity, value-domain, and mutation signals;
- compact, partition, rotate, and place values in situ with a stated invariant;
- derive Kadane's algorithm and return both the maximum sum and its range;
- merge intervals without mutating caller-owned nested arrays;
- use cyclic placement or sign marking only when the value domain proves indexes are safe;
- build prefix and difference arrays with correct half-open boundaries;
- traverse and transform matrices while validating rectangular or square shape;
- distinguish auxiliary space from output space and input mutation; and
- discuss ownership, defensive copying, concurrency, and memory locality.

CHOOSE IT | Array pattern decision map

Ask these questions in order.

Question about the problem	Likely pattern	Critical precondition
Is the output a filtered form of the input?	read/write compaction	mutation and output-prefix contract are allowed
Is the value domain a few categories?	Dutch national flag partition	each value maps to a known category
Is the answer one contiguous range?	window, prefix state, or Kadane	operation and constraints determine which one
Are intervals overlapping after ordering?	sort and sweep	endpoint semantics are defined
Are values drawn from $1..n$?	cyclic placement or sign marking	mutation is allowed and validation is safe
Is the operation a cyclic shift?	reversal rotation	normalize distance and define empty behavior
Are there many immutable range queries?	prefix array	extra $O(n)$ memory is acceptable
Are there many offline range updates?	difference array	final values can be materialized later
Is the input a grid?	row/column/diagonal/boundary traversal	rectangular versus ragged contract is explicit

Two signals can coexist. "Maximum sum after many range additions" might use a difference array to materialize values and Kadane afterward. Pattern selection is composition guided by invariants, not a one-label classification exercise.

Java array semantics and ownership

An array variable stores a reference. Assignment aliases the same array:

JAVA

```
int[] original = {1, 2, 3};
int[] alias = original;
alias[0] = 99;           // original[0] is now 99
int[] copy = original.clone(); // independent primitive array
```

For `int[][]`, `clone()` copies only the outer array. The row arrays remain shared. A defensive deep copy must clone every row. This matters for interval inputs represented as `int[][]`: sorting the outer array reorders caller-visible row references, and editing an endpoint mutates a shared row. A production-quality helper either documents destructive behavior or copies first.

Arrays expose mutable state without synchronization. A method cannot safely scan an array while another thread modifies it unless the caller provides coordination. Declaring a reference `final` prevents rebinding, not element mutation.

Use `Arrays.copyOf` or `clone` deliberately. Copying costs $O(n)$ time and space but creates an ownership boundary. In interview answers, state whether auxiliary-space analysis includes a defensive copy and why the API chooses it.

Pattern 1: stable compaction

To retain selected elements in original order, maintain a read index and a write boundary. For deduplicating a sorted array, retain the first value and write a value only when it differs from the last retained value.

Invariant before reading index `read: [0, write)` contains exactly one representative of each run found in `[0, read)`, in sorted order. Since `write ≤ read`, overwriting `values[write]` cannot destroy unread data. At termination, the logical result is `[0, write)`; the suffix is unspecified.

TRACE IT | Dry run

Input: `[1, 1, 2, 2, 2, 5]`.

read value	action	logical prefix
first 1	initialize	<code>[1]</code>
1	duplicate, skip	<code>[1]</code>
2	write at 1	<code>[1,2]</code>
2	duplicate, skip	<code>[1,2]</code>
2	duplicate, skip	<code>[1,2]</code>
5	write at 2	<code>[1,2,5]</code>

Time is $O(n)$, auxiliary space is $O(1)$, and order is stable. The sortedness precondition is essential; non-adjacent duplicates in an unsorted array would survive.

Pattern 2: Dutch national flag partition

When values belong to three categories, maintain four regions:

TEXT	
<code>[0, low)</code>	settled low category
<code>[low, scan)</code>	settled middle category
<code>[scan, high]</code>	unknown
<code>(high, n)</code>	settled high category

For values 0, 1, and 2:

- if `values[scan] == 0`, swap it with `values[low]`, then increment both;
- if it is `1`, increment `scan`;
- if it is `2`, swap it with `values[high]` and decrement `high`, but do not increment `scan` because the incoming value is unclassified.

The unknown region shrinks every iteration, proving termination. Each element is classified in $O(1)$ work, so time is $O(n)$ and auxiliary space is $O(1)$.

For `[2,0,2,1,1,0]`, the first swap brings `0` from the end to index `0` but leaves an unclassified `0` at `scan`, which must be inspected next. Incrementing `scan` after the high-category swap is the classic bug.

In a production API, use an enum or classifier when categories have domain meaning. Magic integers make validation and evolution harder.

Pattern 3: Kadane's maximum subarray

For each index `i`, define `endingHere` as the best sum of a nonempty subarray ending exactly at `i`. Such a subarray either starts at `i` or extends the best subarray ending at `i - 1`:

TEXT

```
endingHere(i) = max(values[i], endingHere(i - 1) + values[i])
```

The global answer is the maximum `endingHere` seen. This is dynamic programming compressed to constant state.

To return a range, track the start of the current candidate. If extending would be worse than starting fresh, set the candidate start to `i`. Update the best range only when the sum improves; a defined tie policy can prefer the earliest or shortest range.

TRACE IT | Dry run

For `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`:

i/value	best ending here	candidate range	global best
0/-2	-2	<code>[0,1)</code>	-2
1/1	1	<code>[1,2)</code>	1
2/-3	-2	<code>[1,3)</code>	1
3/4	4	<code>[3,4)</code>	4
4/-1	3	<code>[3,5)</code>	4
5/2	5	<code>[3,6)</code>	5
6/1	6	<code>[3,7)</code>	6
7/-5	1	<code>[3,8)</code>	6
8/4	5	<code>[3,9)</code>	6

The answer is sum 6 over `[3,7)`, values `[4,-1,2,1]`. Initializing the best sum to zero would incorrectly allow an empty subarray for all-negative input. Use `long` for accumulated sums.

Pattern 4: sort and merge intervals

An interval contract must say whether endpoints are closed, open, or half-open. The implementation below treats intervals as closed `[start, end]`, so `[1,4]` and `[4,7]` overlap. For half-open intervals, touching at 4 would not overlap unless the product requirement chooses coalescing.

Algorithm:

- 1 validate each interval has two endpoints and `start <= end`;
- 2 copy the intervals to avoid mutating the caller;
- 3 sort by start, then end using comparison methods that cannot overflow;
- 4 keep one current merged interval;
- 5 extend it when the next start is at most the current end; otherwise emit it and begin another.

Invariant: emitted intervals are sorted, pairwise disjoint, and exactly cover all processed intervals except the current merged interval. Time is $O(n \log n)$ due to sorting; the sweep is $O(n)$. Output and defensive-copy space are $O(n)$.

TRACE IT | Dry run

`[[1,3],[2,6],[8,10],[10,12]]` becomes `[[1,6],[8,12]]`. The first two overlap. The third starts a new component. The final interval touches 10 and merges under the closed-interval contract.

Pattern 5: cyclic placement

If an array of length n contains values in the domain $1..n$, value v has a natural home at index $v - 1$.

While a value is in range and not already at home, swap it into its home. Each successful swap settles at least one value, so there are at most $O(n)$ successful swaps even though a `while` appears inside a `for`.

For first missing positive, values outside $1..n$ are irrelevant. After placement, the first index i whose value is not $i + 1$ identifies the missing positive; if all positions match, the answer is $n + 1$.

Input `[3,4,-1,1]` evolves as follows:

- at index 0, place 3 at index 2: `[-1,4,3,1]`;
- index 1 holds 4, place it at index 3: `[-1,1,3,4]`;
- index 1 now holds 1, place it at index 0: `[1,-1,3,4]`;
- index 1 is the first mismatch, so the answer is 2.

Time is $O(n)$, auxiliary space is $O(1)$, and the input is destroyed. Duplicates require the `values[home] != values[i]` guard; without it, equal values swap forever.

Pattern 6: in-place sign marking

For values guaranteed to lie in $1..n$, index $\text{abs}(v) - 1$ can record whether value v has appeared by making that cell negative. If the cell is already negative, v is a duplicate.

This is useful but contract-heavy:

- zero and out-of-range values produce invalid indexes;
- repeated reporting policy must be defined when a value appears more than twice;
- input signs are destroyed unless a restoration pass runs;
- `Math.abs(Integer.MIN_VALUE)` remains negative, though the domain precondition excludes it; and
- a negative-zero marker does not exist, which is why the domain begins at 1.

Cyclic placement is often easier to defend for first-missing-positive. Sign marking is concise for presence or duplicate detection when mutation is explicitly permitted.

Pattern 7: rotation by reversal

To rotate an array right by **distance**:

- 1 `normalize k = floorMod(distance, n);`
- 2 `reverse the whole array;`
- 3 `reverse [0, k);`
- 4 `reverse [k, n).`

For `[1,2,3,4,5,6,7]` and `k = 3`:

TEXT

```
whole reverse: [7,6,5,4,3,2,1]
first part:    [5,6,7,4,3,2,1]
second part:   [5,6,7,1,2,3,4]
```

The result preserves the relative order inside both cyclic segments. Time is $O(n)$ and auxiliary space is $O(1)$. Handle an empty array before taking a modulus by zero. `Math.floorMod` gives intentional behavior for negative distances.

Pattern 8: prefix arrays

For immutable range-sum queries, define a `long` prefix array of length `n + 1`:

TEXT

```
prefix[0] = 0
prefix[i + 1] = prefix[i] + values[i]
```

The sum over half-open range `[left, right)` is `prefix[right] - prefix[left]`. Everything before `left` cancels. Construction takes $O(n)$ time and $O(n)$ space; each query takes $O(1)$.

For `[3,-2,5,1]`, the prefix is `[0,3,1,6,7]`. Range `[1,4)` sums to `7 - 3 = 4`, matching `-2 + 5 + 1`.

Prefix state is a snapshot. If the underlying array changes, the prefix is stale. A system with frequent point updates may need a Fenwick tree or segment tree rather than rebuilding.

Pattern 9: difference arrays for offline range updates

A difference array stores changes between adjacent positions. To add `delta` to half-open range `[left, right)`:

TEXT

```
difference[left] += delta
difference[right] -= delta    // if right is within the sentinel array
```

Taking a running sum materializes all values. Allocate $n + 1$ slots so the subtraction at `right == n` is safe. After q updates, total time is $O(q + n)$ instead of $O(qn)$. Use `long` because updates accumulate.

Invariant while materializing: the running sum equals the net delta applying at the current index. This technique is offline; querying arbitrary points before materialization or interleaving updates and queries calls for a different data structure.

Pattern 10: matrix traversal and transformation

Java matrices may be ragged. A rectangular algorithm must validate row lengths. Row-major traversal normally has better locality because each Java row is a contiguous array. Column-major traversal still takes $O(\text{rows} * \text{cols})$ time but jumps among row objects.

For an in-place 90-degree clockwise rotation, require a square matrix. Transpose across the main diagonal, then reverse every row:

TEXT

```
transpose: matrix[row][col] <--> matrix[col][row] for col > row
reverse:   each row from both ends
```

After transpose, the original column becomes a row; reversal gives the clockwise orientation. Time is $O(n^2)$ and auxiliary space is $O(1)$. Rectangular rotation changes dimensions and cannot be done through the same `int[][]` shape without allocating a new matrix.

Runnable Java 21 reference implementation

The class below compiles as written. Run its checkpoints with `java -ea ArrayPatternToolkit`.

JAVA (continued 1/8)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public final class ArrayPatternToolkit {
    private ArrayPatternToolkit() {}

    public record Subarray(long sum, int start, int endExclusive) {}

    public static int compactSortedUnique(int[] values) {
        requireArray(values);
        if (values.length == 0) {
            return 0;
        }
        int write = 1;
        for (int read = 1; read < values.length; read++) {
            if (values[read] != values[write - 1]) {
                values[write++] = values[read];
            }
        }
        return write;
    }

    public static void dutchFlag(int[] values) {
        requireArray(values);
        int low = 0;
        int scan = 0;
        int high = values.length - 1;
        while (scan <= high) {
            switch (values[scan]) {
                case 0 -> swap(values, low++, scan++);
                case 1 -> scan++;
            }
        }
    }
}
```

JAVA (continued 2/8)

```
        case 2 -> swap(values, scan, high--);
        default -> throw new IllegalArgumentException("expected only 0, 1, and 2");
    }
}

public static Subarray maxSubarray(int[] values) {
    requireArray(values);
    if (values.length == 0) {
        throw new IllegalArgumentException("a nonempty array is required");
    }
    long endingHere = values[0];
    long best = values[0];
    int candidateStart = 0;
    int bestStart = 0;
    int bestEnd = 1;

    for (int i = 1; i < values.length; i++) {
        long extended = endingHere + values[i];
        if (values[i] > extended) {
            endingHere = values[i];
            candidateStart = i;
        } else {
            endingHere = extended;
        }
        if (endingHere > best) {
            best = endingHere;
            bestStart = candidateStart;
            bestEnd = i + 1;
        }
    }
    return new Subarray(best, bestStart, bestEnd);
}

public static int[][] mergeClosedIntervals(int[][] intervals) {
```


JAVA (continued 3/8)

```
    if (intervals == null) {
        throw new IllegalArgumentException("intervals must not be null");
    }
    int[][] copy = new int[intervals.length][2];
    for (int i = 0; i < intervals.length; i++) {
        if (intervals[i] == null || intervals[i].length != 2
            || intervals[i][0] > intervals[i][1]) {
            throw new IllegalArgumentException("invalid interval at index " + i);
        }
        copy[i] = intervals[i].clone();
    }
    Arrays.sort(copy, Comparator.comparingInt((int[] a) -> a[0])
        .thenComparingInt(a -> a[1]));
    List<int[]> merged = new ArrayList<>();
    for (int[] interval : copy) {
        if (merged.isEmpty() || interval[0] > merged.get(merged.size() - 1)[1]) {
            merged.add(interval.clone());
        } else {
            int[] last = merged.get(merged.size() - 1);
            last[1] = Math.max(last[1], interval[1]);
        }
    }
    return merged.toArray(int[][]::new);
}

public static int firstMissingPositive(int[] values) {
    requireArray(values);
    for (int i = 0; i < values.length; i++) {
        while (values[i] >= 1 && values[i] <= values.length) {
            int home = values[i] - 1;
            if (values[home] == values[i]) {
                break;
            }
            swap(values, i, home);
        }
    }
}
```

JAVA (continued 4/8)

```
    }
    for (int i = 0; i < values.length; i++) {
        if (values[i] != i + 1) {
            return i + 1;
        }
    }
    return values.length + 1;
}

public static List<Integer> duplicatesBySignMarking(int[] values) {
    requireArray(values);
    for (int value : values) {
        if (value < 1 || value > values.length) {
            throw new IllegalArgumentException("values must be in 1..n");
        }
    }
    List<Integer> duplicates = new ArrayList<>();
    for (int i = 0; i < values.length; i++) {
        int value = (int) Math.abs((long) values[i]);
        int marker = value - 1;
        if (values[marker] < 0) {
            duplicates.add(value);
        } else {
            values[marker] = -values[marker];
        }
    }
    for (int i = 0; i < values.length; i++) {
        values[i] = (int) Math.abs((long) values[i]);
    }
    return duplicates;
}

public static void rotateRight(int[] values, int distance) {
    requireArray(values);
    if (values.length == 0) {
```

JAVA (continued 5/8)

```
        return;
    }
    int k = Math.floorMod(distance, values.length);
    reverse(values, 0, values.length);
    reverse(values, 0, k);
    reverse(values, k, values.length);
}

public static long[] prefixSums(int[] values) {
    requireArray(values);
    long[] prefix = new long[values.length + 1];
    for (int i = 0; i < values.length; i++) {
        prefix[i + 1] = prefix[i] + values[i];
    }
    return prefix;
}

public static long rangeSum(long[] prefix, int left, int rightExclusive) {
    if (prefix == null || left < 0 || left > rightExclusive
        || rightExclusive >= prefix.length) {
        throw new IllegalArgumentException("invalid prefix range");
    }
    return prefix[rightExclusive] - prefix[left];
}

public static long[] applyRangeAdds(int length, int[][] updates) {
    if (length < 0 || updates == null) {
        throw new IllegalArgumentException("invalid length or updates");
    }
    long[] difference = new long[length + 1];
    for (int i = 0; i < updates.length; i++) {
        int[] update = updates[i];
        if (update == null || update.length != 3) {
            throw new IllegalArgumentException("update must be [left,rightExclusive,delta]");
        }
    }
}
```

JAVA (continued 6/8)

```
        int left = update[0];
        int right = update[1];
        int delta = update[2];
        if (left < 0 || left > right || right > length) {
            throw new IllegalArgumentException("invalid update range");
        }
        difference[left] += delta;
        difference[right] -= delta;
    }
    long[] result = new long[length];
    long running = 0;
    for (int i = 0; i < length; i++) {
        running += difference[i];
        result[i] = running;
    }
    return result;
}

public static void rotateSquareClockwise(int[][] matrix) {
    int n = validateSquare(matrix);
    for (int row = 0; row < n; row++) {
        for (int col = row + 1; col < n; col++) {
            int temporary = matrix[row][col];
            matrix[row][col] = matrix[col][row];
            matrix[col][row] = temporary;
        }
    }
    for (int[] row : matrix) {
        reverse(row, 0, row.length);
    }
}

private static int validateSquare(int[][] matrix) {
    if (matrix == null) {
        throw new IllegalArgumentException("matrix must not be null");
    }
}
```

JAVA (continued 7/8)

```
    }
    int n = matrix.length;
    for (int[] row : matrix) {
        if (row == null || row.length != n) {
            throw new IllegalArgumentException("matrix must be square");
        }
    }
    return n;
}

private static void reverse(int[] values, int from, int toExclusive) {
    for (int left = from, right = toExclusive - 1; left < right; left++, right--) {
        swap(values, left, right);
    }
}

private static void swap(int[] values, int first, int second) {
    int temporary = values[first];
    values[first] = values[second];
    values[second] = temporary;
}

private static void requireArray(int[] values) {
    if (values == null) {
        throw new IllegalArgumentException("values must not be null");
    }
}

public static void main(String[] args) {
    int[] unique = {1, 1, 2, 2, 2, 5};
    int uniqueLength = compactSortedUnique(unique);
    assert Arrays.equals(Arrays.copyOf(unique, uniqueLength), new int[] {1, 2, 5});

    int[] colors = {2, 0, 2, 1, 1, 0};
}
```

JAVA (continued 8/8)

```
dutchFlag(colors);
assert Arrays.equals(colors, new int[] {0, 0, 1, 1, 2, 2});

assert maxSubarray(new int[] {-2, 1, -3, 4, -1, 2, 1, -5, 4})
    .equals(new Subarray(6, 3, 7));
assert maxSubarray(new int[] {-8, -3, -6}).sum() == -3;

int[][] input = {{1, 3}, {2, 6}, {8, 10}, {10, 12}};
int[][] merged = mergeClosedIntervals(input);
assert Arrays.deepEquals(merged, new int[][] {{1, 6}, {8, 12}});
assert Arrays.deepEquals(input, new int[][] {{1, 3}, {2, 6}, {8, 10}, {10, 12}});

assert firstMissingPositive(new int[] {3, 4, -1, 1}) == 2;
int[] marked = {4, 3, 2, 7, 8, 2, 3, 1};
assert duplicatesBySignMarking(marked).equals(List.of(2, 3));
assert Arrays.equals(marked, new int[] {4, 3, 2, 7, 8, 2, 3, 1});

int[] rotated = {1, 2, 3, 4, 5, 6, 7};
rotateRight(rotated, 3);
assert Arrays.equals(rotated, new int[] {5, 6, 7, 1, 2, 3, 4});

long[] prefix = prefixSums(new int[] {3, -2, 5, 1});
assert Arrays.equals(prefix, new long[] {0, 3, 1, 6, 7});
assert rangeSum(prefix, 1, 4) == 4;
assert Arrays.equals(applyRangeAdds(5,
    new int[][] {{1, 4, 3}, {0, 2, -1}},
    new long[] {-1, 2, 3, 3, 0}));

int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
rotateSquareClockwise(matrix);
assert Arrays.deepEquals(matrix,
    new int[][] {{7, 4, 1}, {8, 5, 2}, {9, 6, 3}});
}
```

Complexity and contract table

Pattern	Time	Auxiliary space	Mutates input?
sorted unique compaction	$O(n)$	$O(1)$	yes
Dutch partition	$O(n)$	$O(1)$	yes
Kadane with range	$O(n)$	$O(1)$	no
merge intervals	$O(n \log n)$	$O(n)$ including defensive copy/output	no
cyclic placement	$O(n)$ aggregate	$O(1)$	yes
sign marking	$O(n)$	$O(1)$ excluding duplicate output	temporarily, then restores
reversal rotation	$O(n)$	$O(1)$	yes
prefix construction/query	$O(n) / O(1)$	$O(n)$	no
difference updates/materialization	$O(q + n)$	$O(n)$	no
square matrix rotation	$O(n^2)$	$O(1)$	yes

WATCH OUT | Edge cases and common mistakes

- 1 **Returning a false empty answer.** Kadane for a nonempty subarray must initialize from the first value, not zero.
- 2 **Comparator overflow.** Use `Comparator.comparingInt`, not `(a, b) -> a[0] - b[0]`.
- 3 **Shallow copies.** Cloning `int[][]` does not clone its rows.
- 4 **Ambiguous interval endpoints.** Decide whether touching intervals merge.
- 5 **Cyclic-placement infinite loops.** Check whether the destination already contains the same value.
- 6 **Unvalidated sign indexes.** Sign marking is valid only for its promised domain.
- 7 **Modulus by zero.** Return early for an empty rotation input.
- 8 **Prefix off by one.** A prefix for `n` values has `n + 1` entries and range `[left, right)` uses `prefix[right] - prefix[left]`.
- 9 **Difference sentinel omitted.** Allocate the extra cell or guard `right == n`.
- 10 **Integer accumulation.** Array values may be `int` while sums, counts, and products need `long`.
- 11 **Ragged matrix assumptions.** Validate shape before using a shared column bound.
- 12 **Hidden destructive behavior.** Name in-place methods clearly and document the valid output region.

SDE-2 production follow-ups

- **Ownership and immutability:** expose an immutable result or copy when inputs cross trust boundaries. Internal hot paths may use ownership transfer to avoid copies, but the contract must be explicit.
- **Large inputs:** check proposed result sizes before allocation. An array still uses `int` indexes in Java, and object headers plus alignment affect real memory beyond element counts.
- **Primitive versus boxed data:** `int[]` avoids per-element boxing and normally has better locality than `List<Integer>`. A collection may be preferable when size changes or abstraction matters more than raw layout.
- **Concurrency:** copying can provide a stable snapshot. Without it, callers must not mutate while the algorithm reads.
- **Observability:** record input length, category counts, and elapsed time rather than logging entire arrays that may contain sensitive data.
- **External memory:** data larger than heap may require chunking, memory-mapped files, a database operation, or a streaming algorithm. Not every array solution scales by increasing the heap.
- **Vectorization and libraries:** use `Arrays.sort`, `Arrays.fill`, `System.arraycopy`, and measured library primitives in production. Hand-written loops are justified by custom invariants, not by assuming they are faster.
- **Versioned semantics:** interval endpoints, rotation direction, and duplicate policy belong in API documentation and tests because changing them silently changes results.

TRY IT | Exercises with model checkpoints

Exercise 1: move zeros stably

Move all zero values to the end while preserving nonzero order.

Model checkpoints: use read/write compaction; `[0, write)` contains the nonzero values processed so far; fill `[write, n)` with zeros; time $O(n)$, space $O(1)$; clarify whether negative zero is relevant for the chosen primitive type.

Exercise 2: maximum product subarray

Return the maximum product of a nonempty contiguous range.

Model checkpoints: track both maximum and minimum products ending at each index because a negative value swaps their roles; define overflow behavior; include zeros and all-negative cases; returning indexes requires tracking candidate starts for both states.

Exercise 3: insert and merge one interval

Given sorted, disjoint closed intervals and one new interval, merge in linear time.

Model checkpoints: emit intervals ending before the new start; merge all intervals starting at or before the current merged end; emit the rest; distinguish closed from half-open touching behavior; avoid mutating caller rows.

Exercise 4: product except self

Return, without division, the product of all values except the current one.

Model checkpoints: output first stores prefix products, then multiply by a running suffix; output storage usually does not count as auxiliary space; discuss zero behavior and overflow; reject the claim that $O(1)$ total space includes a required result array.

Exercise 5: set matrix zeroes

If a cell is zero, zero its whole row and column using constant auxiliary space.

Model checkpoints: use first row and first column as markers; preserve separate flags for whether they originally contained a zero; mark before clearing; validate rectangularity; explain why clearing immediately destroys future information.

Exercise 6: range update service

Turn the difference-array helper into a service supporting online updates and queries.

Model checkpoints: a plain difference array cannot answer online prefix values efficiently; consider a Fenwick tree for range-add/point-query or two Fenwick trees for range-add/range-sum; define overflow, synchronization, snapshot, and persistence semantics.

Exercise 7: missing and duplicate pair

An array of length n should contain $1 \dots n$, but one value is missing and another appears twice.

Model checkpoints: cyclic placement gives an intuitive destructive solution; XOR or sum equations require careful derivation and overflow handling; validate the promised contract if this is production data; return a named result rather than an anonymous array.

Interview answer checklist

- ☐ I selected the pattern from a concrete recognition signal.
- ☐ I stated whether ordering, contiguity, or a bounded value domain is required.
- ☐ I identified every settled and unknown array region.
- ☐ I documented whether the method mutates the input and which output region is valid.
- ☐ I widened arithmetic before sums, differences, counts, or products can overflow.
- ☐ I defined empty, singleton, duplicate, and all-negative behavior.
- ☐ I specified interval endpoint semantics and comparator safety.
- ☐ I distinguished primitive arrays, nested-array copies, and collection alternatives.
- ☐ I counted defensive copies and returned output honestly in the space analysis.
- ☐ I can explain how updates, concurrency, and data larger than memory change the design.

RECAP | Summary

Array mastery is a map of invariants. Compaction separates a valid output prefix from unread input. Dutch partition shrinks an unknown region. Kadane carries the best range ending at one position. Interval merging turns sorted overlap into a sweep. Cyclic placement and sign marking borrow array cells as metadata only under strict value-domain and ownership contracts. Reversal implements rotation without extra storage. Prefix and difference arrays trade memory and update timing for constant-time range operations. Matrix transformations add shape and aliasing concerns. The SDE-2 standard is to connect each technique to its proof, boundaries, mutation policy, and production consequences.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: traverse, copy, rotate, and partition
- Interview Core: two pointers, windows, and prefix sums
- SDE-2 Follow-up: defend mutation and memory choices
- Challenge: combine sorting with interval or index placement

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous volume: 05 - Loop Mastery, Patterns, and Index Calculations](#)

[Series index](#)

[Next volume: 07 - Strings and String Problem-Solving Patterns](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 6 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.